

Unit-04

Clipping

4.1 Introduction to Clipping

Clipping is an essential process in computer graphics used to manage which parts of a graphical primitive (such as points, lines, or polygons) are displayed on the screen. The specified region where graphics are visible is called the clipping window or viewing area. Any parts of the primitive that fall outside this boundary are removed or clipped.

Need for Clipping:

- To focus on a specific portion of the scene.
- To improve rendering speed by discarding irrelevant parts.
- To create effects like zooming or panning.
- To fit graphical elements within the display screen.

Clipping Types:

1. **Point Clipping:** Tests whether a point lies inside or outside the clipping window.
 2. **Line Clipping:** Determines the visible portion of a line segment.
 3. **Polygon Clipping:** Clips a polygon to the clipping window, retaining only the visible parts.
-

4.2 Point Clipping

Point Clipping is the process of determining whether a given point (x, y) lies within a specified rectangular region called the clipping window. If the point lies inside the window, it is retained; otherwise, it is discarded.

Purpose of Point Clipping

- To restrict graphical objects or data points to a defined display area.
- To enhance rendering efficiency by ignoring points outside the visible region.

Clipping Window

A rectangular boundary defined by:

- $x_{\min}$: Left boundary (minimum x-coordinate).
- $x_{\max}$: Right boundary (maximum x-coordinate).
- $y_{\min}$: Bottom boundary (minimum y-coordinate).
- $y_{\max}$: Top boundary (maximum y-coordinate).

Condition for Point Clipping

A point (x, y) is retained if:

$$x_{\min} \leq x \leq x_{\max} \text{ and } y_{\min} \leq y \leq y_{\max}$$

If any of these conditions fail, the point is clipped (removed).

4.3 Line Clipping

Line Clipping is the process of removing or modifying portions of a line segment that lie outside a specified rectangular region, known as the clipping window. The goal is to ensure only the visible portion of the line within the window is retained for rendering.

Cohen-Sutherland Line Clipping Algorithm

The Cohen-Sutherland Line Clipping Algorithm is an efficient method for clipping a line segment to a rectangular clipping window. It assigns region codes to endpoints of the line segment to quickly determine whether the line lies inside, outside, or partially inside the clipping window.

Steps of the Algorithm:

1. Assign Region Codes:

Each endpoint of the line is assigned a 4-bit code indicating its position relative to the window ($x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$):

- Bit 1: Above
- Bit 2: Below
- Bit 3: Right

- Bit 4: Left

2. Perform Logical Tests:

- **Trivial Acceptance:** If both endpoints have a region code of 0000, the line is entirely inside the window.
- **Trivial Rejection:** If the logical AND of the region codes of both endpoints is nonzero, the line is entirely outside the window.

3. Clipping Calculation:

- For other cases, calculate the intersection of the line with the clipping boundary and assign new region codes.

Advantages:

- Simple and efficient for basic line clipping.
-

4.4 Polygon Clipping

Polygon clipping is the process of removing parts of a polygon that lie outside the clipping window, ensuring only the visible portion within the window is displayed. This is essential in computer graphics for rendering objects efficiently within a bounded viewing area.

Sutherland-Hodgeman Polygon Clipping Algorithm

The Sutherland-Hodgeman algorithm is an efficient method for clipping a polygon against a rectangular clipping window. It operates by processing each edge of the polygon against one side of the clipping window at a time, producing the visible portion within the defined boundaries.

Steps of the Algorithm:

1. **Input:** The polygon's vertices and the clipping window.
2. **Processing Each Edge:** For each edge of the clipping window:
 - For each polygon edge, determine the relationship of vertices (inside or outside).
 - Perform necessary operations to generate new vertices:
 - **Case 1:** Both endpoints are inside → Retain the endpoint.

- **Case 2:** One endpoint inside, the other outside → Compute and retain the intersection point and the inside endpoint.
- **Case 3:** Both endpoints outside → Discard.

3. **Repeat for All Edges:** Clip against all four edges of the clipping window.

Advantages:

- Efficient for convex and concave polygons.
- Produces accurate results.

Example of Clipping

Problem Statement:

Given a polygon with vertices A (1, 1), B (6, 1), C (6, 6), D (1, 6) and a rectangular clipping window

defined by $x_{\min} = 2$, $x_{\max} = 5$, $y_{\min} = 2$, $y_{\max} = 5$, clip the polygon using the Sutherland-Hodgeman Polygon Clipping Algorithm.

Initial Polygon:

Vertices: A (1, 1), B (6, 1), C (6, 6), D (1, 6)

Clipping Window:

Boundary edges: Left ($x = 2$), Right ($x = 5$), Bottom ($y = 2$), Top ($y = 5$)

Step-by-Step Solution:

Clipping Against the Left Boundary ($x = 2$):

1. Process edge AB: Add intersection point $P_1(2, 1)$ and B (6, 1).
2. Process edge BC: Add C (6, 6).
3. Process edge CD: Add intersection point $P_2(2, 6)$.
4. Process edge DA: No points added.

Output Polygon After Left Boundary: $P_1(2, 1)$, B (6, 1), C (6, 6), $P_2(2, 6)$

Clipping Against the Right Boundary ($x = 5$):

1. Process edge P1B: Add intersection point P3(5, 1).
2. Process edge BC: No points added.
3. Process edge CP2: Add intersection point P4(5, 6) and P2(2, 6).
4. Process edge P2P1: Add P1(2, 1).

Output Polygon After Right Boundary: P3(5, 1), P4(5, 6), P2(2, 6), P1(2, 1)

Clipping Against the Bottom Boundary ($y = 2$):

1. Process edge P3P4: Add intersection point P5(5, 2).
2. Process edge P4P2: Add P2(2, 6).
3. Process edge P2P1: Add intersection point P6(2, 2).
4. Process edge P1P3: No points added.

Output Polygon After Bottom Boundary: P5(5, 2), P4(5, 6), P2(2, 6), P6(2, 2)

Clipping Against the Top Boundary ($y = 5$):

1. Process edge P5P4: Add intersection point P7(5, 5).
2. Process edge P4P2: No points added.
3. Process edge P2P6: Add intersection point P8(2, 5) and P6(2, 2).
4. Process edge P6P5: Add P5(5, 2).

Output Polygon After Top Boundary: P7(5, 5), P8(2, 5), P6(2, 2), P5(5, 2)

Final Clipped Polygon:

Vertices: P7(5, 5), P8(2, 5), P6(2, 2), P5(5, 2)